Pasted text.txt TXT

README (3).md MD

o sa ti dau continutul unui fisier de la laboratorul de AI. saptamana aceasta avem de realizat livrabilul 2, in echipa cu alte 3 persoane. iti dau continutul unui readme care ar reprezenta planul nostru sa alcatuim proiectul. poti sa mi explici mai detaliat te rog cam ce am avea de facut in livrabilul 2? de asemenea, avand in vedere ca munca trebuie impartita la 4 oameni, imi poti spune cam ce ar trebui sa faca fiecare dintre noi si eventual sa imi poti da sugestii de 4 prompturi (pentru fiecare persoana cate un promt care rezolva partea lui de livrabil) prin care putem rezolva acest livrabil?

Repartizarea motorului, analizei și a testelor pe componente

Ce cere Livrabilul 2 (L6)

Din enunț, Deliverable 2 înseamnă să **modelați un automat în sistemul vostru**, cu patru elemente:

1. **Starea lumii și regulile:** cum reprezentați grila (matricea `numpy`) și automatul (spec-ul JSON validat).
2. **Parametrii experimentului:** număr de generații, seed, lumea inițială, dacă se salvează generațiile intermediare.
3. **Stocarea și recuperarea stării:** salvați un experiment și îl reîncărcați identic.
4. **Testare pe 2–3 automate:** sistemul citește generația curentă, aplică regulile și produce generația următoare corectă.

Planul vostru din README pentru L6 se potrivește exact: spec JSON + validator, motor `neighbour_count`, lume inițială manuală/aleatoare, salvare/încărcare, întrebările 1 și 4.

Ce NU intră încă: parserul de limbaj natural (spec-urile se scriu de mână), rețeaua neuronală, interfața completă. Rule 30 și Langton's Ant nu sunt necesare acum.

Ce ar fi bine să arătați la laborator: un script care, pentru Game of Life, Brian's Brain și Forest Fire, încarcă spec-ul, construiește lumea inițială, rulează câțiva pași, afișează generațiile, salvează experimentul, îl reîncarcă și verifică faptul că e identic. Adăugați și testele automate care trec.

Cel mai important lucru: contractul comun

Lucrați în paralel, deci trebuie să fixați **înainte de a scrie cod** numele fișierelor, semnăturile funcțiilor și semantica regulilor. Altfel, la final, piesele nu se vor potrivi. Am scris mai jos un bloc „CONTEXT COMUN" pe care fiecare îl lipește la începutul promptului ei, ca toți asistenții să genereze cod compatibil.

Am extins puțin formatul regulilor față de README, pentru că altfel nu puteți descrie toate automatele:

- condiție cu `min` / `max`, pentru „cel puțin 2 vecini” (Cyclic CA) sau „cel puțin un vecin FIRE” (Forest Fire);
- regulă fără `if`, adică necondiționată (FIRING → DYING la Brian's Brain);
- ordinea regulilor contează: se aplică prima regulă potrivită.

Împărțirea pe persoane

Persoană	Ce face la D2	Livrează
Căprariu Ioana (validare)	modelul `pydantic` al spec-ului, validare cu mesaje clare, spec-urile JSON pentru 5 automate	`src/parser/schema.py`, `specs/*.json`, teste
Fichioș Alexia (motor)	`step()` și `run()` vectorizate, întrebările 1 și 4 (bonus 2 și 5, sunt ușoare)	`src/engine/simulator.py`, `src/analysis/questions.py`, teste
Rendiuc Ioana (lumi inițiale)	lume din text sau fișier, aleatoare cu seed, predefinită; **testele de capăt la capăt pe 3 automate**	`src/world/initial.py`, `tests/test_d2_automata.py`
Basarab Roxana (stocare)	salvare/încărcare experimente (JSON + `.npz` + SQLite), scriptul demo pentru laborator	`src/storage/experiments.py`, `demo_d2.py`

Ordinea: Ioana C. publică cât mai repede `schema.py` și un spec, pentru că restul depind de ele (e un fișier mic, poate fi gata în prima zi). Ceilalți pot începe imediat, pe baza contractului. Testele de integrare (Rendiuc) și demo-ul (Roxana) se rulează după ce motorul ajunge în `main`.

Prompturile

Bloc comun (se lipește la începutul fiecărui prompt)

```
CONTEXT COMUN – proiect CelLab (Python 3.11, numpy, pydantic v2, pytest)

Construim o aplicație de automate celulare, în echipă de 4. Componentele
comunică DOAR prin:

1) SPEC-ul automatului (JSON). Exemplu Game of Life:
{
  "name": "Game of Life",
  "kind": "neighbour_count",
  "grid": {"rows": 50, "cols": 50, "boundary": "toroidal"},
  "states": ["DEAD", "ALIVE"],
  "neighborhood": "moore",
  "rules": [
    {"from": "DEAD",  "to": "ALIVE", "if": {"count": "ALIVE", "in": [3]}},
    {"from": "ALIVE", "to": "DEAD",  "if": {"count": "ALIVE", "not_in": [2,
3]}}
  ],
  "otherwise": "unchanged"
}
- kind: deocamdată doar "neighbour_count".
- grid.boundary: "toroidal" (marginile se leagă) sau "fixed" (în afara grilei
se consideră starea cu codul 0).
- states: lista numelor; CODUL unei stări = indexul ei în listă.
- neighborhood: "moore" (8 vecini) sau "von_neumann" (4 vecini).
- O regulă: {"from": S, "to": T, "if": C (opțional), "probability": p
(opțional, implicit 1.0)}
  C are "count": Z (starea numărată printre vecini) și EXACT UNA dintre
forme:
  {"in": [..]} | {"not_in": [..]} | {"min": a} | {"max": b} | {"min": a,
"max": b}
  Fără "if" = regulă necondiționată.
- Semantică: actualizare SIMULTANĂ (toți citesc generația veche). Pentru
fiecare celulă se aplică PRIMA
  regulă (în ordinea din listă) cu "from" egal cu starea celulei și condiția
adevărată; dacă există
  "probability" < 1, regula se aplică doar cu acea probabilitate (rng = numpy
Generator primit ca parametru).
  Dacă nicio regulă nu se aplică, celula rămâne neschimbată.

2) GRILA: np.ndarray 2D, dtype np.int8, shape (rows, cols), valori = coduri
de stare.
3) GENERAȚIILE: list[np.ndarray], generations[0] = lumea inițială.
```

```
Fișiere și semnături FIXE (nu le schimba numele):
- src/parser/schema.py: class AutomatonSpec (pydantic), parse_spec(data:
dict) -> AutomatonSpec,
  load_spec(path) -> AutomatonSpec, class SpecError(Exception) cu atributul
.errors: list[str],
  metode AutomatonSpec.state_code(name) -> int și AutomatonSpec.to_dict() ->
dict.
  În model, câmpul "from" se numește from_ (alias "from"), "if" se numește
if_ (alias "if").
- src/engine/simulator.py: step(grid, spec, rng=None) -> np.ndarray;
run(grid, spec, n, rng=None) -> list[np.ndarray]
- src/analysis/questions.py: state_after(grid, spec, n, rng=None);
count_cells(grid, spec, state_name) -> int
- src/world/initial.py: from_text(text, spec, symbols=None),
random_world(spec, densities, seed, rows=None, cols=None),
  predefined(name, spec, rows=None, cols=None), to_text(grid, spec,
symbols=None)
- src/storage/experiments.py: ExperimentConfig, save_experiment(...),
load_experiment(exp_id), list_experiments()
Structura: src/{parser,engine,analysis,world,storage}, specs/, tests/.
Fiecare folder din src are __init__.py.
```

1. Căprariu Ioana – spec și validare

```
[CONTEXT COMUN]

Sarcina mea: src/parser/schema.py și spec-urile predefinite.

1. Modelul pydantic v2 AutomatonSpec exact după formatul de mai sus
(folosește aliasuri pentru "from"/"if").
2. Validare SEMANTICĂ, nu doar de tip:
   - nume de stări unice, cel puțin 2 stări;
   - "from", "to" și "count" din fiecare regulă sunt stări existente;
   - condiția are exact una dintre formele permise; valorile sunt între 0 și
numărul maxim de vecini
     (8 la moore, 4 la von_neumann); min <= max;
   - probability în [0, 1]; rows și cols > 0; kind, boundary și neighborhood
au valori permise;
   - avertisment (nu eroare) dacă o regulă nu poate fi atinsă niciodată
pentru că o regulă anterioară
     cu același "from" o acoperă complet.
3. parse_spec strânge TOATE erorile și aruncă SpecError cu mesaje clare în
română care indică regula,
   de ex. „Regula 2: starea 'ALIV' nu există. Stări cunoscute: DEAD, ALIVE.”
4. Scrie în specs/: game_of_life.json, brians_brain.json (OFF/FIRING/DYING,
OFF->FIRING dacă exact 2
   vecini FIRING), forest_fire.json (determinist: FIRE->EMPTY, TREE->FIRE
dacă cel puțin 1 vecin FIRE),
   cyclic_ca.json (4 stări 0..3, x -> (x+1) mod 4 dacă cel puțin 2 vecini au
valoarea următoare),
   crystal_growth.json (EMPTY->CRYSTAL dacă exact 1 vecin CRYSTAL).
5. tests/test_schema.py: toate spec-urile din specs/ se încarcă fără erori,
plus cel puțin 8 spec-uri
   greșite care produc mesajul așteptat.

Explică-mi pe scurt fiecare decizie de validare, ca să o pot prezenta.
```

2. Fichioș Alexia – motor și întrebări

```
[CONTEXT COMUN]

Sarcina mea: src/engine/simulator.py și src/analysis/questions.py.

1. step(grid, spec, rng=None) VECTORIZAT cu numpy, fără bucle pe celule:
   - numărarea vecinilor de o stare Z: np.roll pentru "toroidal", np.pad cu 0
pentru "fixed";
     Moore = 8 deplasări, von Neumann = 4;
   - calculează numărările o singură dată pentru fiecare stare care apare în
reguli;
   - aplică regulile în ordine cu o mască „deja_atribuit”, ca să respecți
regula „prima potrivită câștigă”;
   - probability < 1: rng.random(shape) < p, aplicată doar celulelor
eligibile; dacă rng e None și
     există probabilități, aruncă eroare clară;
   - nu modifica grila primită; returnează o grilă nouă, dtype int8.
2. run(grid, spec, n, rng=None) -> listă cu n+1 generații (inclusiv cea
inițială).
3. questions.py:
   - state_after (întrebarea 1) și count_cells (întrebarea 4);
   - bonus: first_stable(grid, spec, max_steps) -> prima generație egală cu
cea anterioară sau None;
     find_cycle(grid, spec, max_steps) -> (start, lungime) sau None, folosind
un dict de hash-uri
     (hash pe grid.tobytes()); pentru spec-uri cu probabilități, aruncă
eroare explicativă.
4. tests/test_simulator.py, cu spec-uri construite direct în test (dicturi
trecute prin parse_spec):
   blinker oscilează cu perioada 2; blocul 2x2 e stabil; glider-ul după 4
pași e deplasat cu (1,1)
   pe grilă toroidală; Brian's Brain: FIRING->DYING->OFF; diferența toroidal
vs fixed la margine;
   o regulă cu probabilitate dă același rezultat pentru același seed.

Explică-mi cum funcționează numărarea vecinilor cu np.roll, ca să o pot
prezenta.
```

3. Basarab Roxana – stocare și demo

```
[CONTEXT COMUN]

Sarcina mea: src/storage/experiments.py și scriptul demo_d2.py.

1. ExperimentConfig (dataclass sau pydantic): generations, seed (opțional),
store_intermediate (bool),
   store_every (int, implicit 1), init_method ("manual" | "random" |
"predefined"), init_params (dict).
2. save_experiment(spec, initial_grid, config, generations,
base_dir="experiments") -> exp_id:
   - folder experiments/<exp_id>/ cu spec.json (spec.to_dict()), config.json,
generations.npz
     (np.savez_compressed; dacă store_intermediate e False, salvează doar
prima și ultima generație,
     altfel fiecare a store_every-a generație, plus ultima, mereu; păstrează
și indicii generațiilor);
   - un rând în experiments/index.db (SQLite): id, nume automat, dată,
generații, seed, cale.
3. load_experiment(exp_id) -> obiect cu spec (AutomatonSpec), config,
generations și indicii lor.
   list_experiments() -> listă din SQLite, cele mai noi primele.
4. tests/test_storage.py: round-trip (salvez, încarc, grilele sunt identice
cu np.array_equal),
   store_every funcționează, un experiment aleator re-rulat cu același seed
dă aceleași generații.
   Folosește tmp_path.
5. demo_d2.py (pentru prezentare): pentru Game of Life, Brian's Brain și
Forest Fire din specs/:
   încarcă spec-ul, creează lumea inițială (from_text sau predefined),
rulează 10 pași, afișează în
   terminal primele 3 generații cu to_text, răspunde la „câte celule de tip X
după N pași”, salvează
   experimentul, îl reîncarcă și afișează „Reîncărcare OK” dacă e identic.

Explică-mi de ce .npz + SQLite și cum aș regăsi un experiment vechi.
```

4. Rendiuc Ioana – lumi inițiale și teste pe automate

```
[CONTEXT COMUN]

Sarcina mea: src/world/initial.py și testele de capăt la capăt pe 3 automate.

1. from_text(text, spec, symbols=None): un rând de text = un rând de grilă;
spațiile dintre caractere
   sunt ignorate. Simboluri implicite: '.' pentru starea 0, iar pentru
celelalte prima literă a numelui
   (ALIVE->A, FIRING->F, DYING->D, TREE->T, FIRE->F...). Dacă două stări au
aceeași literă, cere
   symbols explicit cu eroare clară. Verifică: rânduri de lungimi egale,
simboluri necunoscute (cu
   rândul și coloana în mesaj). from_file(path, spec, symbols=None) citește
același format.
2. random_world(spec, densities, seed, rows=None, cols=None): densities =
{"ALIVE": 0.3}; restul
   devine starea 0; suma > 1 sau stare necunoscută -> eroare; folosește
np.random.default_rng(seed),
   ca același seed să dea aceeași lume. rows/cols implicit din spec.grid.
3. predefined(name, spec, ...): "single_center", "glider", "blinker",
"empty".
4. to_text(grid, spec, symbols=None): inversa lui from_text
(from_text(to_text(g)) == g).
5. tests/test_world.py pentru toate cele de mai sus.
6. tests/test_d2_automata.py (cerința principală a livrabilului): încarcă
specs/game_of_life.json,
   specs/brians_brain.json, specs/forest_fire.json și verifică pas cu pas, cu
grile scrise ca text:
   - Game of Life: blinker și glider;
   - Brian's Brain: exemplul ". . F . . / . F . F . / . . . . ." pentru 2
generații, calculate de mână;
   - Forest Fire: exemplul din enunț "T T T T . / T T . T T / . T F T T / T T
T T T" până nu mai
     există FIRE; verifică numărul de TREE rămase și că stă stabil.
   Pune rezultatele așteptate scrise explicit în test, nu calculate tot cu
motorul.

Explică-mi cum am verificat de mână rezultatele așteptate.
```

Câteva sfaturi

- **Faceți împreună, 15 minute, contractul înainte de a începe.** Dacă vreți să schimbați ceva în blocul comun (de exemplu numele unei funcții), schimbați-l pentru toți.

- **Rezultatele așteptate din teste verificați-le de mână**, nu doar cu ce a generat asistentul. Un test greșit care „trece” vă poate costa la prezentare.
- **Salvați fiecare conversație cu asistentul în** `ai_conversations/`, de exemplu `D2_alexia_motor.md`. E obligatoriu la predare și se vede ușor cine a contribuit cu ce, iar notele pot diferi între membri.
- **Cereți asistentului să explice codul**, așa cum e scris la finalul fiecărui prompt. La prezentare fiecare trebuie să poată explica partea ei.

Dacă vreți, vă pot ajuta să adăugați în README o secțiune „Deliverable 2” cu ce s-a livrat și cum se rulează demo-ul.

Faceți împreună, 15 minute, contractul înainte de a începe. Dacă...

poti sa ma ajuti te rog cu contractul si sa mi-l stabilesti tu ca sa il pot impartasi cat mai repede cu coechipierii mei?

Checking which document connector tools are available ▸

Stabilesc eu contractul complet și îl pun într-un document pe care îl poți trimite direct colegelor.

Used Claude Docs: Guide ▸

Am pus structura documentului; acum completez secțiunile, începând cu regulile contractului.

Used Claude Docs, used artifacts ▸

CelLab – Contract comun D2 e gata. Îl poți partaja colegelor, edita direct sau comenta.

Față de prompturile de data trecută am schimbat două lucruri, ca să rămână compatibile cu contractul. Simbolurile pentru lumea text stau acum în spec (câmpul `symbols`), deci `from_text` și `to_text` nu mai primesc parametrul `symbols`. În plus, `save_experiment` primește toate generațiile și decide singură ce păstrează. În prompturi, înlocuiți blocul „CONTEXT COMUN” cu secțiunile 3–6 din document.

poti sa mi zici in ce ordine trebuie fiecare dintre noi sa lucreze si cine depinde de taskul cui? ca sa ne putem apuca

Weighing whether a dependency diagram is worth drawing ▸

Pe scurt: **Ioana C. și Roxana încep primele, pentru că toate celelalte depind de ele.** Ioana C. livrează `schema.py`, iar Roxana scheletul de foldere. Apoi toate lucrați în paralel, iar la final Rendiuc și Roxana leagă totul.

Cine de cine depinde

Task	Depinde de	Îl așteaptă
Schelet repo (Roxana)	nimic	toată lumea
`schema.py` minimal (Ioana C.)	schelet	Alexia, Rendiuc, Roxana
Validare completă + spec-uri (Ioana C.)	`schema.py` minimal	testele lui Rendiuc
Motor + întrebări (Alexia)	`schema.py`	testele lui Rendiuc, demo-ul
Lumi inițiale (Rendiuc)	`schema.py`	testele lui Rendiuc, demo-ul
Stocare (Roxana)	`schema.py`	demo-ul
Teste pe 3 automate (Rendiuc)	motor + lumi + spec-uri	—
`demo_d2.py` (Roxana)	tot	—

Cheia: Ioana C. nu trebuie să termine toată validarea ca să le deblocheze pe celelalte. Ajunge varianta minimă: modelele `pydantic`, `parse_spec`, `load_spec`, `state_code`, `symbol_map`, `is_stochastic`, `to_dict`, plus `game_of_life.json`. Ideal ar fi în `main` mâine. Validarea semantică detaliată și mesajele de eroare le adaugă după, fără să blocheze pe nimeni.

Ordinea pentru fiecare

Căprariu Ioana

1. `schema.py` minimal + `game_of_life.json` → în `main` cât mai repede
2. celelalte 4 spec-uri (Brian's Brain și Forest Fire primele, de ele are nevoie Rendiuc)
3. validarea semantică completă + `lint_spec`
4. `tests/test_schema.py`

Basarab Roxana

1. scheletul repo: foldere, `__init__.py`, `requirements.txt`, `pyproject.toml` (durează 15 minute, primul lucru)
2. `ExperimentConfig` și logica „ce generații păstrez", care nu depind de nimic
3. după `schema.py`: `save_experiment` / `load_experiment` / SQLite + teste (generațiile din teste le poate face de mână cu `np.zeros`, fără motor)

4. la final, `demo_d2.py`, după ce motorul și lumile sunt în `main`

Fichioș Alexia

1. `count_neighbours`, care nu are nevoie de spec și poate fi scris imediat, pe matrici simple
2. după `schema.py`: `step` și `run` + teste (blinker, block, glider)
3. `state_after`, `count_after`
4. bonus: `first_stable`, `find_cycle`

Rendiuc Ioana

1. până apare `schema.py`: calculează **de mână** rezultatele așteptate pentru testele finale (Brian's Brain 2 generații, Forest Fire până se stinge). E partea cea mai lentă și nu depinde de cod.
2. după `schema.py`: `from_text`, `to_text`, `random_world`, `predefined` + `tests/test_world.py`
3. scrie `tests/test_d2_automata.py` cu rezultatele de la pasul 1
4. îl rulează când motorul ajunge în `main` și raportează Alexiei ce nu trece

Cronologic

1. **Azi/mâine:** Roxana face scheletul, apoi Ioana C. pune `schema.py` minimal. Între timp Alexia scrie `count_neighbours`, Rendiuc calculează rezultatele de mână, iar Roxana scrie `ExperimentConfig`.
2. **Mijlocul săptămânii:** toate patru lucrează în paralel pe modulele lor.
3. **Cu o zi înainte de laborator:** se unesc pull request-urile, Rendiuc rulează testele pe 3 automate, Roxana rulează demo-ul, iar ce nu merge se repară împreună.

Dacă vreți, pot adăuga această ordine în document, la secțiunea „Ordinea de lucru și Git".

ok pai o sa ti dau promptul meu, contextul il stii deja,[CONTEXT COMUN]

Sarcina mea: src/engine/simulator.py și src/analysis/questions.py.

1. step(grid, spec, rng=None) VECTORIZAT cu numpy, fără bucle pe celule:
   - numărarea vecinilor de o stare Z: np.roll pentru "toroidal", np.pad cu 0 pentru "fixed"; Moore = 8 deplasări, von Neumann = 4;
   - calculează numărările o singură dată pentru fiecare stare care apare în reguli;
   - aplică regulile în ordine cu o mască „deja_atribuit", ca să respecți regula „prima potrivită câștigă";
   - probability < 1: rng.random(shape) < p, aplicată doar celulelor eligibile; dacă rng e None și există probabilități, aruncă eroare clară;
   - nu modifica grila primită; returnează o grilă nouă, dtype int8.
2. run(grid, spec, n, rng=None) -> listă cu n+1 generații (inclusiv cea inițială).
3. questions.py:
   - state_after (întrebarea 1) și count_cells (întrebarea 4);

- bonus: first_stable(grid, spec, max_steps) -> prima generație egală cu cea anterioară sau None; find_cycle(grid, spec, max_steps) -> (start, lungime) sau None, folosind un dict de hash-uri (hash pe grid.tobytes()); pentru spec-uri cu probabilități, aruncă eroare explicativă.

4. tests/test_simulator.py, cu spec-uri construite direct în test (dicturi trecute prin parse_spec): blinker oscilează cu perioada 2; blocul 2x2 e stabil; glider-ul după 4 pași e deplasat cu (1,1) pe grilă toroidală; Brian's Brain: FIRING->DYING->OFF; diferența toroidal vs fixed la margine; o regulă cu probabilitate dă același rezultat pentru același seed.

Explică-mi cum funcționează numărarea vecinilor cu np.roll, ca să o pot prezenta. mai ai nevoie de ceva vreun material sa iti dau scris de vreo colega?

Show more

Drafting a temporary stub to keep tests running ▸

Nu am nevoie de nimic în plus: lucrez direct după contract. Ca testele să ruleze până apare `schema.py` de la Ioana C., scriu și o variantă minimă temporară a ei, pe care o veți înlocui cu fișierul ei.

Created 6 files, shared 3 files, ran 4 commands ▸

Nu am avut nevoie de nimic de la colege. Am lucrat direct după contract și am scris partea ta: `simulator.py`, `questions.py` și testele lor. Toate cele 26 de teste trec, inclusiv întrebările bonus 2 și 5.

Atenție la două fișiere din arhivă care nu sunt ale tale:

- `src/parser/schema.py` este o **variantă temporară**, ca să poți rula testele până pune Ioana C. versiunea ei. Nu o urca în `main`. Când apare a ei, o ștergi pe asta, iar testele tale ar trebui să treacă neschimbate, dacă și ea respectă contractul.
- `pyproject.toml` e al Roxanei; i l-am pus doar ca să meargă `pytest`. Spune-i ce conține (3 rânduri), ca să-l includă în al ei.

Cum funcționează numărarea vecinilor cu `np.roll`

Ideea: în loc să iei fiecare celulă și să te uiți la cei 8 vecini ai ei, **muți toată grila** pe rând în cele 8 direcții și aduni rezultatele.

Pasul 1, masca. Pentru starea numărată, de exemplu ALIVE, faci o matrice cu 1 unde celula e ALIVE și 0 în rest: `mask = (grid == code)`.

Pasul 2, deplasarea. `np.roll(mask, shift=(1, 0), axis=(0, 1))` mută toată matricea cu un rând în jos. Ce iese pe margine intră pe partea opusă, exact ca la grila toroidală. După mutare, în poziția fiecărei celule se află valoarea **vecinului ei de deasupra:**

```
mask                    roll cu (1, 0)        = „vecinul de sus e viu?”
. . . .                 . . . .
. A . .      ──►         . . . .
. . . .                 . A . .     ← celula de sub A „vede” un vecin de sus
viu
```

Pasul 3, suma. Faci asta pentru toate cele 8 deplasări (−1, 0, +1 pe rând și coloană, fără (0, 0)) și aduni cele 8 matrici. În fiecare poziție rezultă numărul de vecini ALIVE ai acelei celule. Sunt doar 8 operații pe toată grila, indiferent cât de mare e, de aceea e mult mai rapid decât buclele Python.

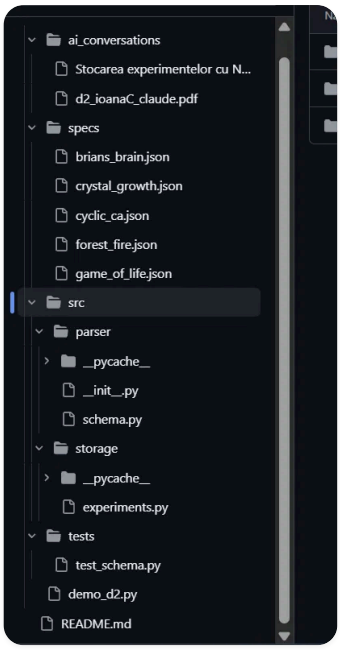
Margine fixă. `np.roll` lipește marginile, deci la `fixed` nu merge. Acolo bordez grila cu un rând sau o coloană în plus (`np.pad`) și iau 8 felii deplasate cu câte o poziție. Un detaliu de menționat la prezentare: contractul spune că în afara grilei e starea 0. De aceea, când număr vecinii de stare 0 (DEAD, EMPTY), bordura se umple cu 1, adică celulele din afară se numără. La celelalte stări bordura e 0.

Alte trei lucruri bune de explicat la prezentare

- **„Prima regulă potrivită câștigă”:** țin o mască `assigned` cu celulele care au primit deja o regulă. Fiecare regulă se aplică doar celulelor neatribuite încă.
- **Actualizare simultană:** condițiile se verifică mereu pe grila **veche**, iar rezultatul se scrie într-o copie. Așa, o celulă schimbată acum nu influențează vecinii în același pas.
- **Ciclurile:** salvez amprenta (hash-ul) fiecărei generații într-un dicționar `amprentă → generație` . Când o amprentă reapare, ciclul începe la generația salvată, iar lungimea lui e diferența. Testul glider-ului e un exemplu frumos: pe o grilă 8×8 revine în același loc după 32 de pași.

Nu uita să salvezi conversația asta în `ai_conversations/D2_alexia_motor.md` .

Download all



imi poti explica putin flowul codului?

Comparing folder positions in the screenshot's layout ▸

Sigur. Îți explic întâi unde intră codul tău în proiect, apoi ce se întâmplă în interiorul lui, pas cu pas.

Unde intră codul tău

```
specs/game_of_life.json ──▶ load_spec()  ──▶ spec (AutomatonSpec)
[Ioana C.]
lumea inițială          ──▶ from_text()  ──▶ grid (matrice int8)
[Rendiuc]
                                    │
                                    ▼
                  run(grid, spec, n) ──▶ [gen0, gen1, …, genN]        [TU:
simulator.py]
                                    │
                ┌───────────────────┴──────────────────┐
                ▼                                      ▼
   save_experiment(...)  [Roxana]        state_after / count_after /
                                         first_stable / find_cycle     [TU:
questions.py]
```

Motorul tău primește doar două lucruri: **spec-ul** (ce reguli are automatul) și **grila** (matricea de numere). Nu știe nimic despre fișiere, text sau interfață. Tocmai de aceea poate fi testat separat.

`step()` : o generație nouă

Să luăm Game of Life cu regulile „DEAD → ALIVE dacă are 3 vecini ALIVE” și „ALIVE → DEAD dacă nu are 2 sau 3 vecini ALIVE”.

1. **Verificări.** `_check_grid` se asigură că grila e 2D și conține doar coduri valide (0 sau 1 aici). Dacă spec-ul are probabilități și nu s-a dat `rng`, oprește execuția cu eroare.
2. **Pregătire.** Se creează trei obiecte:
   - `new` : o copie a grilei, în care se scrie rezultatul. Grila veche rămâne neatinsă.
   - `assigned` : o matrice de `False`. Marchează celulele care au primit deja o regulă.
   - `counts_cache` : un dicționar gol, pentru numărările de vecini.
3. **Bucla pe reguli** (nu pe celule!). Pentru fiecare regulă, în ordine:
   - a. `eligible = (grid == src) & ~assigned` : celulele care au starea `from` și n-au primit încă o regulă. La prima regulă: toate celulele DEAD.
   - b. **Condiția.** Dacă regula are `if` , se numără vecinii de tipul cerut cu `count_neighbours` , o singură dată per stare datorită cache-ului. Apoi `_condition_mask` transformă condiția (`in` , `not_in` , `min` , `max`) într-o mască de `True` / `False` , iar `eligible` păstrează doar celulele unde condiția e adevărată.
   - c. **Probabilitatea.** Dacă `probability < 1` , se trage câte un număr aleator pentru fiecare celulă, iar cele care „pică” sunt scoase din `eligible` . Ele rămân neatribuite, deci pot fi prinse de regula următoare.
   - d. **Aplicarea.** `new[eligible] = dst` scrie starea nouă, iar `assigned |= eligible` le marchează ca folosite.
4. **Restul celulelor** rămân cum erau, pentru că `new` a pornit ca o copie a grilei (asta e `otherwise: "unchanged"`).

Detaliul important: condițiile se calculează mereu pe `grid` (generația veche), nu pe `new` . Așa, toate celulele se actualizează „deodată”.

`run()`

Pornește cu `[copie a grilei]` și apelează `step` de `n` ori, punând fiecare rezultat în listă. Întoarce `n+1` generații.

`questions.py`

Funcție	Ce face
`state_after`	apelează `step` de n ori și păstrează doar ultima grilă (nu ține tot istoricul în memorie)
`count_cells`	`np.count_nonzero(grid == cod)`: câte celule au acel cod
`count_after`	`state_after`, apoi numărare
`first_stable`	merge pas cu pas; prima generație k identică cu k+1 e răspunsul
`find_cycle`	ține un dicționar `hash → generație`; când un hash reapare, întoarce `(start, lungime)`

Ultimele două verifică întâi că automatul nu are probabilități. Dacă are, explică printr-o eroare de ce nu se poate răspunde.

Ce am observat în repo, din captură

- `tests/` și `demo_d2.py` par să fie în `src/`; în contract ele sunt la rădăcină, lângă `README.md`. Dacă rămân acolo, importurile `from src.…` din teste pot să nu meargă.
- `__pycache__` e urcat pe GitHub. Ar trebui pus un `.gitignore` cu `__pycache__/`, `.pytest_cache/` și `experiments/`.
- Lipsesc `src/__init__.py` și `src/storage/__init__.py`. Python le tolerează de obicei, dar contractul le cere.

Când urci `src/engine/` și `src/analysis/`, folosește `schema.py`-ul Ioanei, nu varianta mea temporară, și rulează `pytest`. Dacă vreun test pică, trimite-mi eroarea și ne uităm împreună.